PROGRAMACIÓN III

Introducción y Fundamentos de JavaScript

¿Qué es JavaScript?

JavaScript es un lenguaje de programación interpretado y dinámico, creado originalmente para agregar interactividad a las páginas web. Es el lenguaje que da vida a las páginas modernas, permitiendo que dejen de ser estáticas y respondan en tiempo real a las acciones del usuario sin necesidad de recargar el sitio.

Aunque nació en el navegador, hoy su uso se ha extendido a servidores (backend), aplicaciones móviles y otros entornos.

El Trío Fundamental de la Web

Para entender el rol de JavaScript, podemos usar la analogía de una casa:

1. **HTML (Estructura):** Son las paredes, puertas y ventanas. Define el contenido y la organización.
2. **CSS (Estilo):** Es la decoración, pintura y muebles. Define el diseño visual.
3. **JavaScript (Comportamiento):** Es la electricidad y la tecnología. Hace que las luces se enciendan o las puertas se abran; aporta la interactividad.

Historia y Evolución

- **Origen:** Fue creado en 1995 por Brendan Eich en Netscape en solo 10 días.
- **Nombre:** Pasó de llamarse "Mocha" a "LiveScript" y finalmente "JavaScript" por estrategia de marketing, aunque no tiene relación técnica con Java.
- **Estandarización:** En 1997 se convirtió en un estándar oficial llamado **ECMAScript**.

Versiones Clave [11]

- **ES5 (2009):** Gran adopción, mejoras en arrays y objetos.
- **ES6 (2015):** Una revolución en el lenguaje. Introdujo const, let, funciones flecha, clases y módulos.
- **Actualidad:** Se lanzan versiones anuales con nuevas optimizaciones.

¿Cómo funciona?

JavaScript corre directamente en el navegador gracias a un **motor** que interpreta el código. Cada navegador tiene el suyo:

- **V8:** Google Chrome y Edge.
- **SpiderMonkey:** Mozilla Firefox.
- **JavaScriptCore:** Safari.

Tipos de Datos y Variables

En JavaScript, no necesitamos indicar el tipo de dato al declarar una variable (tipado dinámico); el lenguaje lo determina según el valor asignado.

Tipos de Datos Primitivos

Son valores simples e inmutables (se almacenan directamente en memoria):

1. **String:** Texto o cadenas de caracteres.
2. **Number:** Incluye tanto enteros como decimales.
3. **Boolean:** Solo dos valores: true o false. Esencial para tomar decisiones.
4. **Undefined:** Valor de una variable declarada pero sin contenido asignado.
5. **Null:** Valor para indicar intencionalmente que una variable está vacía.

Nota: Los primitivos se comparan por valor. Si copias uno a otra variable, se crea una copia independiente.

Declaración de Variables

Existen tres formas de declarar variables, pero es vital entender sus diferencias:

Palabra Clave	Alcance (Scope)	Redeclaración	Uso Recomendado
var	Global o de función	Permitida	**Evitar**. Causa errores confusos (hoisting) y está en desuso.
let	De bloque (llaves {})	No permitida	Usar para valores que cambiarán (ej. contadores).
const	De bloque (llaves {})	No permitida	**Opción por defecto**. No se puede reasignar, aunque sí es un objeto/array, su contenido interno sí puede cambiar.

Buenas Prácticas

1. Usar **const** por defecto para evitar reasignaciones accidentales.
2. Usar **let** solo si el valor necesita cambiar.
3. Usar nombres descriptivos (ej: usuarioActivo en vez de x) para que el código sea legible.

📘 Operadores en JavaScript

Los operadores son símbolos que nos permiten realizar cálculos, comparar valores y tomar decisiones.

1. Operadores Aritméticos

Se usan para cálculos matemáticos. JavaScript respeta el orden matemático (paréntesis, multiplicación/división, suma/resta)[34343434].

- + Suma.
- - Resta.
- * Multiplicación.
- / División (devuelve decimales si no es exacta)[35].
- % **Módulo:** Devuelve el resto de la división. Muy útil para saber si un número es par (resto 0) o impar[36363636].

2. Operadores de Comparación

Evalúan relaciones y **siempre devuelven un booleano** (true / false).

- Mayor/Menor: >, <, >=, <=.
- **Igualdad Estricta (===):** Compara **valor y tipo de dato**. Es la recomendada.
  - Ej: 5 === "5" es false.
- Igualdad No Estricta (==): Convierte los tipos automáticamente, lo que puede generar errores.
- Desigualdad Estricta (!==): Verifica si los valores o tipos son diferentes.

3. Operadores Lógicos

Combinan condiciones:

- **AND (&&):** Devuelve true solo si **ambas** condiciones son verdaderas.
- **OR (||):** Devuelve true si **al menos una** condición es verdadera.
- **NOT (!):** Invierte el valor lógico (de true a false y viceversa).

4. Operador Ternario

Es una forma concisa de escribir un if/else en una sola línea para asignaciones simples.

- **Sintaxis ¿ condición** ? valor_si_ verdadero : valor_si_falso.

📘 Estructuras de Control (Condicionales y Bucles)

Estas estructuras controlan el flujo del programa, permitiendo tomar decisiones o repetir tareas.

Condicionales

if / else if / else

Evalúa una condición (debe ser booleana). Si es verdadera, ejecuta el bloque de código.

- **else if:** Para evaluar múltiples condiciones en secuencia.
- **else:** Se ejecuta si ninguna de las anteriores se cumplió.

switch

Útil para comparar una sola variable contra muchas opciones (casos).

- Usa igualdad estricta.
- Requiere break para evitar ejecutar los casos siguientes.
- default funciona como opción por defecto.

Bucles (Ciclos)

1. Bucle for

Ideal cuando sabemos cuántas veces repetir algo. Tiene 3 partes en una línea: inicialización, condición y actualización.

2. Bucle while

Repite una acción mientras la condición sea verdadera. Útil cuando no sabemos el número exacto de iteraciones.

- ⚠ Cuidado**:** La condición debe volverse falsa eventualmente para evitar bucles infinitos.

3. Bucle do while

Similar al while, pero **garantiza que el código se ejecute al menos una vez** antes de evaluar la condición.

Iteradores Modernos

for...of

Ideal para recorrer elementos iterables como **Arrays** o **Strings** de forma simple, sin manejar índices manuales.

forEach

Es un método de los Arrays. Ejecuta una función para cada elemento. Es más legible y expresivo que un bucle tradicional.

- Recibe: el elemento actual, el índice (opcional) y el array completo (opcional).
- **Nota:** No se puede usar break o continuar en un forEach.

PROGRAMACIÓN III

Funciones en JavaScript

Las funciones son uno de los bloques de construcción más importantes en la programación. Nos permiten encapsular código para reutilizarlo tantas veces como sea necesario, haciendo nuestros programas más ordenados y fáciles de mantener.

1. ¿Qué es una función?

Una función es un conjunto de instrucciones que realiza una tarea específica o calcula un valor. Para usarla, primero debemos **declarar y** luego **invocar** (llamarla).

2. Tipos de Declaración

A. Declaración de Función (Function Declaration)

Es la forma clásica. Tienen la ventaja de tener *hoisting*, es decir, se pueden llamar antes de ser declaradas en el código.

JavaScript
```
function saludar(nombre) {
   return `Hola, ${nombre}!`;
}

console.log(saludar("Pedro")); // Salida: "Hola, Pedro!"
```

B. Expresión de Función (Function Expression)

Aquí guardamos la función dentro de una variable (const o let). No tienen *hoisting*.

JavaScript
```
const sumar = function(a, b) {
   return a + b;
};
```

C. Funciones Flecha (Arrow Functions) - ES6

Introducidas en 2015, son una sintaxis más corta y moderna. Son muy utilizadas hoy en día, especialmente en frameworks como React.

- Si la función tiene una sola línea de código, el return es implícito (no hace falta escribirlo).

JavaScript
```
// Sintaxis moderna
const multiplicar = (a, b) => a * b;

console.log(multiplicar(5, 2)); // Salida: 10
```

3. Parámetros y Argumentos

- **Parámetros:** Son las variables que definimos en la declaración de la función (ej: a y b).
- **Argumentos:** Son los valores reales que pasamos al ejecutar la función (ej: 5 y 2).

Programación Orientada a Objetos (POO)

JavaScript es un lenguaje basado en prototipos, pero desde ES6 incorporó la sintaxis de **Clases** (class), lo que facilita mucho la escritura de código orientado a objetos, haciéndolo similar a lenguajes como Java o C#.

1. Clases y Objetos

- **Clase:** Es el "plano" o molde. Define las propiedades y comportamientos que tendrán los objetos.
- **Objeto:** Es la instancia creada a partir de la clase (el producto final).

2. Estructura de una Clase

JavaScript
```
class Usuario {
   // El constructor se ejecuta al crear el objeto
   constructor(nombre, email) {
      this.nombre = nombre;
      this.email = email;
   }

   // Método (comportamiento)
   saludar() {
      return `Hola, soy ${this.nombre}`;
   }
}

// Instanciación (Crear el objeto)
const alumno = new Usuario("Santiago", "santiago@utn.edu.ar");

console.log(alumno.saludar()); // Salida: "Hola, soy Santiago"
```

3. Conceptos Clave

- **Constructor:** Un método especial que inicializa las propiedades del objeto.
- **this:** Hace referencia al objeto actual que está ejecutando el código.
- **Herencia (extends):** Permite crear una clase nueva basada en una existente, heredando sus propiedades y métodos.

JavaScript
```
class Admin extends Usuario {
   eliminarCuenta() {
      console.log("Cuenta eliminada");
   }
}
```

Manipulación del DOM y Eventos

El **DOM** (Document Object Model) es la representación del HTML en forma de árbol que el navegador crea en memoria. JavaScript utiliza el DOM para hacer que la página sea dinámica.

1. Selección de Elementos

Para modificar algo, primero debemos "capturarlo" del HTML.

- document.getElementById('id'): Selecciona por ID.
- document.querySelector('.clase'): Selecciona el primer elemento que coincida (usando sintaxis de CSS).
- document.querySelectorAll('p'): Selecciona todos los elementos que coincidan.

2. Manipulación

Una vez seleccionado, podemos cambiar su contenido o estilo.

JavaScript
```
const titulo = document.querySelector('h1');

// Cambiar texto
titulo.textContent = "Nuevo Título desde JS";

// Cambiar estilos
titulo.style.color = "blue";

// Agregar/Quitar clases CSS
titulo.classList.add('destacado');
```

3. Eventos

Los eventos son acciones que ocurren en el sistema (clics, teclas presionadas, envío de formularios). Usamos addEventListener para "escuchar" estos eventos.

JavaScript
```
const boton = document.querySelector('#miBoton');

boton.addEventListener('click', () => {
   alert("¡Hiciste clic!");
});
```

LocalStorage y SessionStorage

La API de **Web Storage** nos permite guardar datos en el navegador del usuario de forma sencilla (clave-valor). Es útil para persistir sesiones, carritos de compras o preferencias (modo oscuro/claro).

1. Diferencias Principales

Característica	**LocalStorage**	**SessionStorage**
Duración	Persiste indefinidamente (hasta que se borre manual).	Se borra al cerrar la pestaña o navegador.
Alcance	Disponible en todas las pestañas del mismo origen.	Solo disponible en la pestaña actual.
Capacidad	Aprox 5MB-10MB.	Aprox 5MB.

2. Métodos Principales (Iguales para ambos)

Se usa el objeto global localStorage o sessionStorage.

- setItem('clave', 'valor'): Guardar un dato.
- getItem('clave'): Recuperar un dato.
- removeItem('clave'): Borrar un dato específico.
- clear(): Borrar todo.

3. El truco del JSON

El Web Storage **solo guarda texto (strings)**. Si queremos guardar objetos o arrays, debemos convertirlos usando JSON.

JavaScript

```
const usuario = { nombre: "Luciano", rol: "Profe" };

// 1. Guardar (Convertir Objeto -> String)
localStorage.setItem('usuarioActivo', JSON.stringify(usuario));

// 2. Recuperar (Convertir String -> Objeto)
const datosGuardados = localStorage.getItem('usuarioActivo');
const usuarioObjeto = JSON.parse(datosGuardados);

console.log(usuarioObjeto.nombre); // "Luciano"
```

PROGRAMACIÓN III

Asincronía y Promesas

JavaScript es un lenguaje **"Single Thread"** (de un solo hilo), lo que significa que solo puede ejecutar una tarea a la vez. Sin embargo, en el desarrollo web necesitamos realizar tareas que tardan tiempo (como pedir datos a un servidor) sin que la pantalla se congele. Aquí es donde entra la **asincronía**.

1. ¿Qué es una Promesa?

Una **Promesa** (Promise) es un objeto que representa la terminación (o el fracaso) eventual de una operación asíncrona y su valor resultante.

Piensa en una promesa como pedir una pizza por delivery:

1. Haces el pedido (**Pendiente / Pending**): No tienes la pizza, pero tienes la "promesa" de que llegará.
2. Mientras esperas, puedes hacer otras cosas (poner la mesa, elegir una película).
3. Si el delivery llega (**Resuelta / Fulfilled**): Tienes tu pizza (el dato).
4. Si ocurre un accidente y no llega (**Rechazada / Rejected**): Recibes una explicación del error.

Estados de una Promesa

- **Pending:** Estado inicial, no se ha cumplido ni rechazado.
- **Fulfilled (Resolved):** La operación se completó con éxito.
- **Rejected:** La operación falló.

2. Async y Await (La forma moderna)

Introducidos en ES2017 (ES8), `async` y `await` son "azúcar sintáctico" sobre las promesas. Nos permiten escribir código asíncrono que **parece** síncrono, haciéndolo mucho más fácil de leer y entender.

- **`async`:** Se coloca antes de una función. Indica que esa función siempre devolverá una promesa.
- **`await`:** Solo se puede usar dentro de una función `async`. Hace que JavaScript "espere" hasta que la promesa se resuelva antes de continuar con la siguiente línea de código.

JavaScript

```
// Función asíncrona
async function obtenerDatos() {
    try {
        // Pausamos la ejecución aquí hasta que la promesa se resuelva
        const resultado = await operacionLenta();
        console.log(resultado);
    } catch (error) {
        // Si la promesa falla, caemos aquí
        console.error("Hubo un error:", error);
    }
}
```

Consumo de APIs con Fetch

Para que nuestras aplicaciones web sean útiles, generalmente necesitan interactuar con un servidor para obtener o guardar datos. Para esto utilizamos la **Fetch API**, que es el estándar moderno nativo de los navegadores.

1. ¿Qué es Fetch?

`fetch()` es un método global que inicia un proceso de recuperación de recursos de la red. Devuelve una **Promesa** que se resuelve en el objeto `Response` representando la respuesta a tu petición.

2. El proceso de "Dos Pasos"

Al usar Fetch, es importante entender que obtener los datos generalmente requiere dos `await`:

1. **Primer `await`:** Esperar a que el servidor conteste (recibimos los encabezados y el estado, pero no necesariamente todo el cuerpo del mensaje).
2. **Segundo `await`:** Convertir esa respuesta al formato que necesitamos (generalmente JSON).

Ejemplo de petición GET (Obtener datos)

JavaScript
```
async function cargarUsuarios() {
    try {
        // 1. Hacemos la petición
        const respuesta = await fetch('https://api.ejemplo.com/usuarios');

        // 2. Verificamos si la respuesta fue exitosa (Status 200-299)
        if (!respuesta.ok) {
            throw new Error('Error al cargar los datos');
        }

        // 3. Convertimos el cuerpo a JSON
        const datos = await respuesta.json();

        console.log(datos); // Aquí ya tenemos el array de usuarios
    } catch (error) {
        console.error(error);
    }
}
```

3. Peticiones POST (Enviar datos)

Por defecto, `fetch` hace una petición **GET**. Si queremos crear un recurso nuevo (como registrar un usuario), debemos configurar un segundo parámetro con las opciones: el método, los encabezados (headers) y el cuerpo (body).

JavaScript
```
async function crearUsuario(nuevoUsuario) {
    try {
        const respuesta = await fetch('https://api.ejemplo.com/usuarios', {
            method: 'POST', // Indicamos que vamos a enviar datos
            headers: {
                'Content-Type': 'application/json' // Avisamos que enviamos JSON
            },
            body: JSON.stringify(nuevoUsuario) // Convertimos el objeto JS a texto JSON
        });

        if (respuesta.ok) {
            console.log("Usuario creado con éxito");
        }
    } catch (error) {
        console.error("Error de red:", error);
    }
}
```

PROGRAMACIÓN III

Trabajo Práctico Integrador: JavaScript

OBJETIVO GENERAL

Transformar la maqueta estática del proyecto "Food Store" (desarrollada previamente en HTML y CSS) en una aplicación dinámica. El alumno deberá eliminar el contenido estático ("hardcodeado") del HTML y sustituirlo por una generación automática de elementos mediante **JavaScript**, aplicando conceptos de Arrays, Objetos, Funciones y manipulación del DOM.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
Arrays y Objetos	Simulación de una base de datos local. Se utilizará un Array de Objetos para almacenar la información de los productos (nombre, precio, imagen, etc.) y un Array de Strings para las categorías.
DOM (Document Object Model)	Selección de elementos contenedores (nodos padres) en el HTML (`getElementById` o `querySelector`) donde se inyectará el contenido.
Template Strings	Uso de comillas invertidas (`) para insertar variables de JavaScript dentro de estructuras HTML de forma limpia y legible.
Bucles (Iteradores)	Uso de métodos como `.forEach()` o `.map()` para recorrer los listados de productos/categorías y generar el HTML correspondiente por cada ítem.

Concepto	Aplicación en el proyecto
Separación de Responsabilidades	Entender que el HTML define la estructura vacía, el CSS el estilo, y JS el contenido y la lógica.

Caso Práctico

Actualmente, el archivo **`index.html`** contiene todos los productos y categorías escritos manualmente. Esto hace que actualizar el catálogo sea una tarea tediosa y propensa a errores. Se requiere automatizar este proceso mediante dos scripts principales.

```
/root
  ├── index.html
  ├── css/
  │   └── styles.css
  ├── js/
  │   ├── data.js  (Aquí irán los arrays de datos)
  │   └── main.js  (Aquí irá la lógica de renderizado)
  └── assets/      (Tus imágenes)
```

PASO 1: Preparar el HTML (`index.html`)

PASO 1: Preparar el archivo index.html

1. **Reutiliza tu base:** Toma el `index.html` que entregaste en el integrador anterior.
2. **Identifica los contenedores:** Busca la sección donde listaste tus productos y la lista de categorías.
3. **Limpia el contenido:** Borra todos los productos y categorías manuales. Deja las etiquetas "padre" (como `<main>`, `<section>` o `<ul>`) completamente vacías.
4. **Asigna IDs:** Asegúrate de que esos contenedores tengan un `id` para que JavaScript sepa exactamente dónde "escribir" la información.
   +2

Ejemplo de cómo debe quedar tu HTML:

HTML

```
<header>
    <h1>🍔 Food Store</h1>
    <nav>
        <a href="#">Inicio</a>
        <a href="#">Mis Pedidos</a>
        <a href="#">Carrito</a>
        <a href="admin.html">Panel Admin</a>
    </nav>
</header>
<aside>
    <h2>Categorías</h2>
    <ul id="lista-categorias"></ul>
</aside>
<main>
    <form>
 <input type="text" placeholder="Buscar productos...">
        <button type="submit">Buscar</button>
    </form>
    <h2>Productos Destacados</h2>
    <section id="contenedor-productos"></section>
</main>
<footer>
<p>© 2024 Food Store. Todos los derechos reservados.</p>
    <p>Contacto: info@foodstore.com</p>
</footer>
```

```
    <script src="./js/data.js"></script>

    <script src="./js/main.js"></script>
```

PASO 2: Definir los Datos (`js/data.js`)

En este archivo simularás la base de datos. Debes crear:

1. Un Array de Strings para las categorías.
2. Un Array de Objetos para los productos (mínimo 4 productos).

Código Guía:

JavaScript

```
// Array de categorías

const categorias = ["Hamburguesas", "Pizzas", "Papas Fritas", "Bebidas"];

const productos = [

  {

    id: 1,

    nombre: "Hamburguesa Triple",

    descripcion: "Triple carne, cheddar y bacon",

    precio: 25000,

    imagen: "https://via.placeholder.com/300x200/ff6347/ffffff?text=Hamburguesa",

    categoria: "Hamburguesas"

  },

  {

    id: 2,

    nombre: "Pizza Muzzarella",

    descripcion: "Salsa casera y orégano",

    precio: 18000,

    imagen: "https://via.placeholder.com/300x200/ff6347/ffffff?text=Pizza",

    categoria: "Pizzas"

  },

];
```

PASO 3: Lógica de Renderizado (`js/main.js`)

Desarrolla las funciones para "pintar" los datos en la pantalla.

A. Renderizar Categorías

Debes crear una función llamada cargarCategorias() que realice lo siguiente:

- Seleccione el elemento del DOM donde se insertarán las categorías (el <ul> con id "lista-categorias").
- Recorra el array de categorías definido en tu archivo de datos.
- Por cada categoría, debe crear un elemento de lista (<li>) que contenga un enlace (<a>).
- Inyecte cada nuevo elemento dentro del contenedor correspondiente.

B. Renderizar Productos

Desarrolla la función cargarProductos() siguiendo estos requerimientos técnicos:

- **Iteración:** Utiliza un método de array (como forEach) para procesar tu lista de productos.
- **Creación de Estructura:** Por cada producto, debes generar un nodo <article> y asignarle la clase CSS necesaria para que mantenga el estilo de tu proyecto anterior.
- **Uso de Plantillas:** Utiliza **Template Strings** (comillas invertidas ` `) para maquetar el contenido interno del article.
- **Contenido Dinámico:** La estructura debe incluir la imagen, el nombre, la descripción y el precio, accediendo a las propiedades del objeto producto.
- **Interactividad:** El botón de "Agregar" debe mostrar un mensaje (puedes usar un alert) que indique el nombre del producto seleccionado.

Requisitos:

- **Variables:** Utilizar `const` para declarar arrays y funciones.
- **Semántica:** El HTML generado por JavaScript debe respetar las etiquetas `<article>`, `<h3>`, `<p>` y `<button>` definidas en el TP de HTML.
- **Rutas:** Las imágenes deben visualizarse correctamente. Recuerda que las rutas relativas en el array de productos se resuelven desde la ubicación del `index.html`.
- **Validación:** No debe haber errores en la consola del navegador (F12).

CONCLUSIONES ESPERADAS

Al finalizar este Trabajo Práctico, se espera que el alumno haya logrado:

- **Comprender la separación de responsabilidades:** Entender que el HTML define la estructura, el CSS la apariencia y JavaScript el contenido y el comportamiento.
- **Independizar los datos de la vista:** Asimilar que la información (productos, categorías) no debe estar "atada" al código HTML, sino residir en estructuras de datos (Arrays/Objetos) que permiten su fácil modificación y escalabilidad.
- **Dominar el flujo de renderizado:** Visualizar cómo JavaScript actúa como intermediario, tomando datos crudos y transformándolos en elementos visuales del DOM (Nodos HTML) en tiempo de ejecución.
- **Automatizar tareas repetitivas:** Valorar la utilidad de los bucles (`forEach`) para generar múltiples elementos repetitivos (como tarjetas de productos) escribiendo el código una sola vez, reduciendo errores y facilitando el mantenimiento.
- **Preparar la lógica para el futuro:** Dejar el proyecto listo para, en la siguiente etapa, reemplazar el archivo `data.js` local por una petición real a una API externa sin tener que modificar la estructura visual del sitio.